

Snakes and Ladders Project Report

1) Report Scope

This report explains the full project in easy English.

It covers:

- * File-by-file purpose
- * Core Java concepts used
- * Why each code block exists
- * How runtime files are produced
- * How launcher works
- * How to run and test

Project folder: D:\PROJECTS\gg

Report date: 2026-04-13

2) Project Goal

The project builds a playable Snakes and Ladders desktop game using Java Swing.

The game focuses on two goals:

1. Make the game visually nice and interactive.
2. Teach Java concepts with simple comments and practical code.

3) File Inventory and Use

3.1 Source files written by us

1. SnakesAndLaddersGame.java
 - * Main source code.
 - * Contains all classes in one file for easy beginner learning.
 - * Includes gameplay, graphics, save/load, threading, annotation, reflection.
2. launch_game.bat
 - * One-click launcher for Windows.
 - * Checks Java compiler availability.
 - * Compiles and runs the game.

3.2 Runtime output files created by the game

1. saved_game.dat
 - * Binary serialized save file.
 - * Stores full game state object.
2. game_state.txt

- * Human-readable text snapshot of current game state.

3. game_log.txt

- * Appends gameplay log messages with timestamp.

3.3 Compiled class files

Files like GameWindow.class, Board.class, Player.class are generated by javac.

- * They are bytecode output.
- * JVM runs these class files.
- * They are not edited manually.

4) Full Analysis of SnakesAndLaddersGame.java

This single file contains multiple classes to keep the project beginner-friendly.

4.1 Entry Point and Safe GUI Startup

Code snippet:

```
[code block start]
public class SnakesAndLaddersGame {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            try {
                GameWindow window = new GameWindow();
                window.setVisible(true);
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(null,
                    "Unable to start the game: " + ex.getMessage(),
                    "Error",
                    JOptionPane.ERROR_MESSAGE);
            }
        });
    }
}
[code block end]
```

Explanation:

- * 'main' starts the program.
- * 'SwingUtilities.invokeLater' ensures GUI code runs on Swing Event Dispatch Thread.
- * 'try/catch' gives safe startup error handling.

4.2 Enum Concept

Code snippet:

```
[code block start]
enum CellType {
    NORMAL,
    LADDER,
    SNAKE
}
[code block end]
```

Explanation:

- * Enum is fixed set of constants.
- * Cleaner than using random strings or numbers.
- * Used to know what happens when player lands on a cell.

4.3 Custom Annotation Concept

Code snippet:

```
[code block start]
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD, ElementType.FIELD})
@interface EasyNote {
    String value();
}
[code block end]
```

Explanation:

- * This creates custom annotation named 'EasyNote'.
- * Runtime retention means reflection can read it while app runs.
- * Target says it can be placed on class, method, and field.

4.4 Serialization Data Classes

Code snippet:

```
[code block start]
class PlayerSnapshot implements Serializable {
    private static final long serialVersionUID = 1L;
    public final String name;
    public final int position;
}

class GameState implements Serializable {
    private static final long serialVersionUID = 1L;
    public final List<PlayerSnapshot> playerSnapshots;
    public final int currentPlayerIndex;
    public final String savedAt;
}
[code block end]
```

Explanation:

- * 'Serializable' allows saving object data as bytes.
- * 'serialVersionUID' is version ID for serialization compatibility.
- * 'GameState' is top-level snapshot used by save/load feature.

4.5 Interface + Implementation

Code snippet:

```
[code block start]
interface Dice {
    int roll();
}

class RandomDice implements Dice {
    private final Random random = new Random();
    @Override
    public int roll() {
```

```

        return random.nextInt(6) + 1;
    }
}

```

[code block end]

Explanation:

- * Interface defines behavior contract.
- * Class implements interface.
- * This gives flexibility: later we could replace dice logic without changing game logic.

4.6 Abstraction + Inheritance + Polymorphism

Code snippet:

```

[code block start]
abstract class Token {
    protected final Color color;
    public abstract void draw(Graphics2D g2, int x, int y, int size);
}

class CircleToken extends Token {
    @Override
    public void draw(Graphics2D g2, int x, int y, int size) {
        // draw token graphics
    }
}
[code block end]

```

Explanation:

- * Abstract class cannot be directly created.
- * Child class provides real drawing implementation.
- * '@Override' is polymorphism in action.

4.7 Encapsulation with Player class

Code snippet:

```

[code block start]
class Player {
    private final String name;
    private final Token token;
    private int position;

    public int getPosition() { return position; }
    public void setPosition(int position) { this.position = position; }
}
[code block end]

```

Explanation:

- * Private fields hide internal data.
- * Getters/setters control read/write.
- * This is core encapsulation.

4.8 Collections + Condition + Switch in Board

Code snippet:

```

[code block start]
private final Map<Integer, Integer> ladders = new HashMap<>();
private final Map<Integer, Integer> snakes = new HashMap<>();

```

```

public CellType getCellType(int cell) {
    if (ladders.containsKey(cell)) return CellType.LADDER;
    else if (snakes.containsKey(cell)) return CellType.SNAKE;
    return CellType.NORMAL;
}

public int getDestination(int cell) {
    return switch (getCellType(cell)) {
        case LADDER -> ladders.get(cell);
        case SNAKE -> snakes.get(cell);
        case NORMAL -> cell;
    };
}

```

[code block end]

Explanation:

- * HashMap stores snake/ladder start->end mapping.
- * 'if/else' decides type.
- * Modern switch expression returns final destination.

4.9 Graphics and Painting in BoardPanel

Code snippet:

```

[code block start]
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g.create();
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
    drawCells(g2, x0, y0, cellSize);
    drawLadders(g2, x0, y0, cellSize);
    drawSnakes(g2, x0, y0, cellSize);
    drawPlayers(g2, x0, y0, cellSize);
    g2.dispose();
}

```

[code block end]

Explanation:

- * Custom painting happens in 'paintComponent'.
- * Antialiasing makes visuals smooth.
- * Board, snakes, ladders, and tokens are drawn in layers.

4.10 GUI Controls in GameWindow

Main control buttons:

- * Roll Dice
- * Restart
- * Save Game
- * Load Game
- * Show Concepts

Event wiring snippet:

```

[code block start]
rollButton.addActionListener(e -> playTurn());
restartButton.addActionListener(e -> restartGame());
saveButton.addActionListener(e -> saveGameAsync());

```

```
loadButton.addActionListener(e -> loadGameAsync());
conceptsButton.addActionListener(e -> showAnnotatedConcepts());
[code block end]
```

Explanation:

- * Each button calls one clear method.
- * Lambda style keeps code short and readable.

4.11 Game Logic and Animation

Key flow:

1. Current player rolls dice.
2. If move crosses 100, move is rejected.
3. Step-by-step token animation runs with Swing Timer.
4. After move, snake/ladder correction happens.
5. Win check runs.
6. Turn changes.

Animation snippet:

```
[code block start]
javax.swing.Timer timer = new javax.swing.Timer(250, null);
timer.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        if (remaining[0] > 0) {
            player.setPosition(player.getPosition() + 1);
            remaining[0]--;
            boardPanel.repaint();
        }
        if (remaining[0] == 0) {
            timer.stop();
            afterMove(player);
        }
    }
});
[code block end]
```

Explanation:

- * Timer updates movement every 250 ms.
- * Gives clear visual movement instead of instant jump.

4.12 File I/O (Text and Binary)

Binary save snippet:

```
[code block start]
try (ObjectOutputStream out = new ObjectOutputStream(
    Files.newOutputStream(SAVE_FILE,
        StandardOpenOption.CREATE,
        StandardOpenOption.TRUNCATE_EXISTING,
        StandardOpenOption.WRITE))) {
    out.writeObject(state);
}
[code block end]
```

Text save snippet:

```
[code block start]
try (BufferedWriter writer = Files.newBufferedWriter(TEXT_STATE_FILE,
    StandardCharsets.UTF_8,
```

```

        StandardOpenOption.CREATE,
        StandardOpenOption.TRUNCATE_EXISTING,
        StandardOpenOption.WRITE)) {
    writer.write("Saved At: " + state.savedAt);
}
[code block end]
Load snippet:
[code block start]
try (ObjectInputStream in = new ObjectInputStream(Files.newInputStream(SAVE_FILE))) {
    Object obj = in.readObject();
    return (GameState) obj;
}
[code block end]

```

Explanation:

- * Binary file stores actual object graph.
- * Text file gives human-readable snapshot.
- * Load reads object back and restores board positions.

4.13 Multithreading and Concurrency

Concurrency objects:

```

[code block start]
private final ExecutorService ioExecutor = Executors.newSingleThreadExecutor();
private final ScheduledExecutorService autoSaveExecutor = Executors.newSingleThreadScheduledExecuto
r();
[code block end]

```

Auto-save snippet:

```

[code block start]
autoSaveExecutor.scheduleAtFixedRate(() -> {
    try {
        GameState snapshot = createSnapshot();
        writeBinaryState(snapshot);
    } catch (Exception ignored) {}
}, 20, 20, TimeUnit.SECONDS);
[code block end]

```

Background save/load snippet:

```

[code block start]
ioExecutor.submit(() -> {
    GameState snapshot = createSnapshot();
    writeBinaryState(snapshot);
    writeTextState(snapshot);
    SwingUtilities.invokeLater(() -> appendLog("Game saved..."));
});
[code block end]

```

Explanation:

- * File work happens in background thread to keep UI responsive.
- * UI updates return to EDT with 'SwingUtilities.invokeLater'.
- * Auto-save runs repeatedly every 20 seconds.

4.14 Reflection

Quick reflection snippet:

```

[code block start]
Field playersField = GameWindow.class.getDeclaredField("players");
playersField.setAccessible(true);

```

```
List<?> list = (List<?>) playersField.get(this);
appendLog("Reflection check: player count = " + list.size());
[code block end]
```

Annotation scan snippet:

```
[code block start]
Method[] methods = GameWindow.class.getDeclaredMethods();
for (Method method : methods) {
    EasyNote note = method.getAnnotation(EasyNote.class);
    if (note != null) {
        appendLog("Method: " + method.getName() + " => " + note.value());
    }
}
[code block end]
```

Explanation:

- * Reflection reads class metadata at runtime.
- * It can access fields/methods and annotation values.
- * Useful for frameworks and tooling.

4.15 Continuous Log File Writing

Snippet:

```
[code block start]
Files.writeString(LOG_FILE, line, StandardCharsets.UTF_8,
    StandardOpenOption.CREATE, StandardOpenOption.APPEND);
[code block end]
```

Explanation:

- * Every log message shown in UI is also written to file asynchronously.
- * This gives persistent trace of gameplay events.

5) Full Analysis of launch_game.bat

File content:

```
[code block start]
@echo off
setlocal

echo -----
echo   Snakes and Ladders - Java Launcher
echo -----

where javac >nul 2>nul
if errorlevel 1 (
    echo ERROR: JDK not found.
    echo Please install Java JDK and add it to PATH.
    pause
    exit /b 1
)

echo Compiling...
javac SnakesAndLaddersGame.java
if errorlevel 1 (
    echo.
    echo Compile failed. Fix errors, then run this launcher again.
```

```

        pause
        exit /b 1
    )

    echo Starting game...
    java SnakesAndLaddersGame

    echo.
    echo Game closed.
    pause
[code block end]

```

Line-by-line easy explanation:

- * Line 1: Hide command echo for cleaner output.
- * Line 2: Start local environment scope.
- * Lines 4-6: Print launcher heading.
- * Line 8: Check if 'javac' exists in PATH.
- * Lines 9-14: If javac missing, show error and stop.
- * Line 16: Show compile message.
- * Line 17: Compile Java source.
- * Lines 18-23: If compile fails, show message and stop.
- * Line 25: Show run message.
- * Line 26: Launch Java program.
- * Lines 28-30: Wait so terminal does not close instantly.

6) Concept Mapping Table

1. OOP classes and objects

- * Classes: Player, Board, BoardPanel, GameWindow, etc.
- * Objects created with 'new' inside constructors and methods.

2. Encapsulation

- * Private fields in Player and controlled access methods.

3. Inheritance

- * CircleToken extends Token.

4. Abstraction

- * Token abstract class.

5. Interface

- * Dice interface and RandomDice implementation.

6. Enum

- * CellType.

7. Collections

- * List and Map usage across game state and board mappings.

8. Exception handling

- * try/catch used in startup, play turn, save/load, reflection.

9. GUI/Event handling

- * Swing components and ActionListener lambdas.

10. Graphics

- * Custom painting in BoardPanel using Graphics2D.

11. File I/O

- * Binary and text save methods plus log file append.

12. Serialization

- * PlayerSnapshot and GameState implement Serializable.

13. Multithreading/concurrency

- * ExecutorService, ScheduledExecutorService, Swing EDT coordination.

14. Annotations

- * Custom EasyNote annotation used on class/method/field.

15. Reflection

- * Runtime field/method/annotation inspection.

7) End-to-End Runtime Flow

1. User opens launcher or runs java command.
2. Main class starts GUI on EDT.
3. GameWindow builds board and side panel.
4. User rolls dice and token animates.
5. Snakes/ladders adjust destination.
6. Logs update UI and file.
7. Save/load works in background thread.
8. Auto-save writes periodic binary snapshot.
9. Show Concepts button scans annotations by reflection.
10. On close, thread pools are shutdown.

8) How to Run

Option A (one click):

- * Double-click 'launch_game.bat'

Option B (manual commands):

- * 'javac SnakesAndLaddersGame.java'
- * 'java SnakesAndLaddersGame'

9) Testing Checklist

- * App opens without error dialog.
- * Roll Dice moves players correctly.

- * Snake and ladder jumps happen as expected.
- * Winner is detected at cell 100.
- * Save Game creates 'saved_game.dat' and 'game_state.txt'.
- * Load Game restores positions and turn.
- * 'game_log.txt' keeps appending logs.
- * Show Concepts writes annotation/reflection info to log panel.

10) Final Summary

This project is a strong teaching project because it combines:

- * Basic Java syntax and logic
- * OOP design
- * Swing GUI and graphics
- * Data persistence
- * Concurrency
- * Runtime metadata features (annotation + reflection)

It is self-contained and beginner-readable with comments and practical features.